SOFTWARE ENGINEERING / OPERATING SYSTEMS

BANKER OS

SMART RESOURCE ALLOCATION USING BANKER'S ALGORITHM

Project Report | OSSP Hackathon - Problem Statement 13

Academic Year 2026-27 • Department of CSE • KLH Hyderabad



Submitted by

Name	Roll number	Team
Nithish S	2520030346	G-13
Sai Sathwik	2520030328	G-13
Ch. Vivek	2520030532	G-13

Submission date: 17 October 2026. Implementation: local-first Linux C engine with a browser dashboard and a terminal fallback.

Academic integrity note: This is a user-input-driven educational resource manager. It simulates resource allocation decisions; it does not change Linux kernel scheduling or actual process resources.

1 Executive Summary

Banker OS is an interactive resource-allocation simulator built to check whether resource requests can be granted without leaving an operating system model in an unsafe state. Instead of relying on hardcoded matrices, the application accepts the number of processes, resource types, Available resources, Allocation matrix, and Maximum-demand matrix directly from users. It derives Need, evaluates a safe sequence, and grants or denies resource requests accordingly.

The project includes a standard C11 algorithm engine, a local Python HTTP adapter using the compiled C shared library, a responsive offline browser dashboard, a terminal application, persistent user workspace storage, data export/import, automated tests and operation history. The central Banker's Algorithm is executed by the C engine in browser mode.

2 Problem Statement and Objectives

Assigned problem: "Smart Resource Allocation Using Banker’s Algorithm". Develop a resource manager that checks resource requests and determines whether the system remains in a safe state.

- Support user-defined processes and resource types rather than fixed example data.
- Compute and display Available, Allocation, Maximum, Need and Total resources.
- Find one safe sequence when such a sequence exists.
- Reject requests exceeding a process’s Need or system Available.
- Tentatively allocate then roll back requests that lead to an unsafe state.
- Support release, updated state, validation, audit history and state restoration.

3 Background and Theoretical Basis

Deadlock is a condition where waiting processes cannot make progress because required resources are held by others. Banker’s Algorithm is a deadlock-avoidance method: a resource request may be committed only if the resulting state is safe. A safe state is a state in which some order exists allowing every process to complete. An unsafe state is not necessarily already deadlocked.

For process Pi and resource Rj: Need[i][j] = Maximum[i][j] - Allocation[i][j]. Total[j] = Available[j] + sum of allocated units for all processes. Work starts equal to Available and increases when simulated process completions release their allocations.

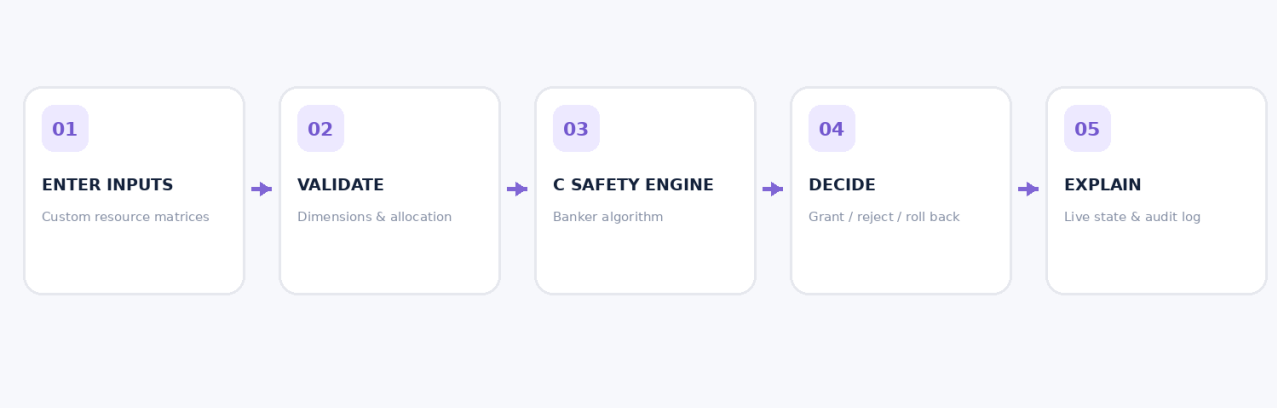
4 Required Inputs and Assumptions

Parameter	Description
n	1-20 processes, labeled P0 to P(n-1)
m	1-10 independently counted resource types, labeled R1 to Rm
Available[m]	Free resource units currently held by the system
Allocation[n][m]	Resources assigned to each process
Maximum[n][m]	Maximum claim for each process and resource

Request[m]	Additional resources requested by a selected process

Model assumptions: non-negative integral resource units; resource capacities remain constant during requests/releases; Max >= Allocation; declared maximum claims do not exceed total system resources; completed processes release their allocated resources during the safety simulation. The system evaluates an abstract resource model, not actual Linux process memory or I/O.

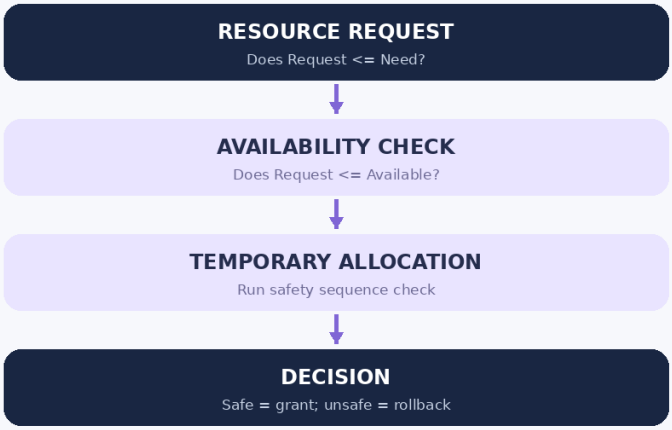
5 System Architecture



Layer A - User interface: web/index.html, style.css and app.js present resource inputs, safety status, visual matrices, request/release controls and an activity log. Layer B - local server: web/server.py handles input validation, state persistence and HTTP endpoints bound to 127.0.0.1. Layer C - C engine: src/bankers_core.c implements validation, safety checking, resource requests and releases. The terminal fallback, src/bankers_cli.c, also provides a Linux command-line learning path.

Module	Main responsibility
Web dashboard	Input wizard, live safe sequence, request/release UI, history and JSON export
Python local adapter	Maps validated JSON vectors to the compiled C shared library
C Bankers core	Evaluate safety and commit/roll back requests
Workspace persistence	runtime/session.json stores the last user-created workspace
Testing	unittest checks numerical, error, conservation and rollback behavior

6 Algorithm Design



6.1 Safety check

- Initialize Work = Available and Finish[i] = false for every process.
- Find an unfinished process for which Need[i][j] <= Work[j] for all resources.
- Simulate completion, add Allocation[i] back into Work, mark the process finished and append Pi to the sequence.
- Repeat until all processes finish, or no eligible process remains.
- If all processes finish, the state is safe; otherwise it is unsafe.

6.2 Resource-request handling

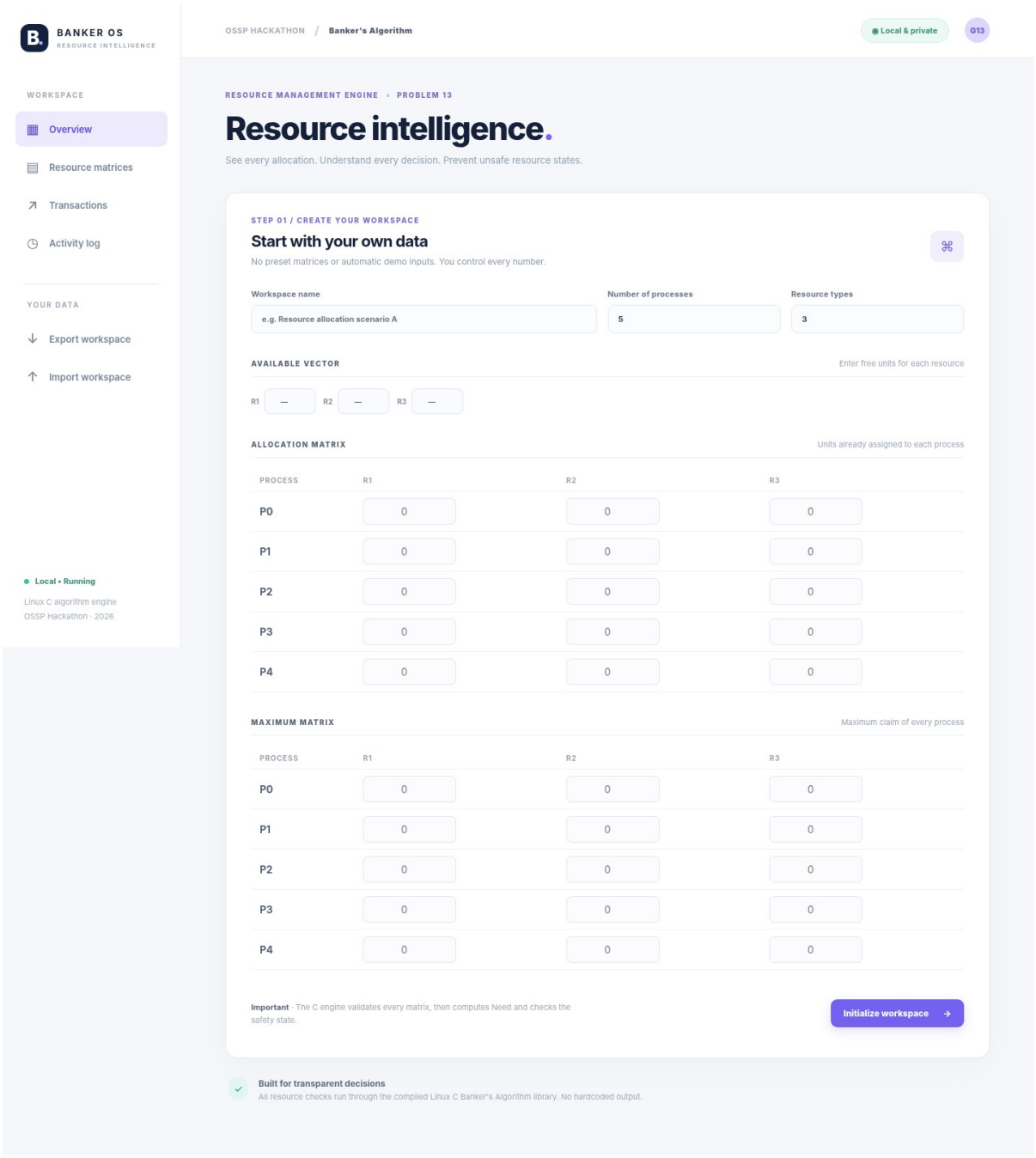
- Check request vector dimensions, selected process, and non-negative quantities.
- If Request > Need, deny; if Request > Available, deny.
- Tentatively update Available and Allocation.
- Run the safety algorithm. Commit if safe; otherwise restore Available and Allocation exactly.
- Report the reason and add the operation to the history.

6.3 Release handling

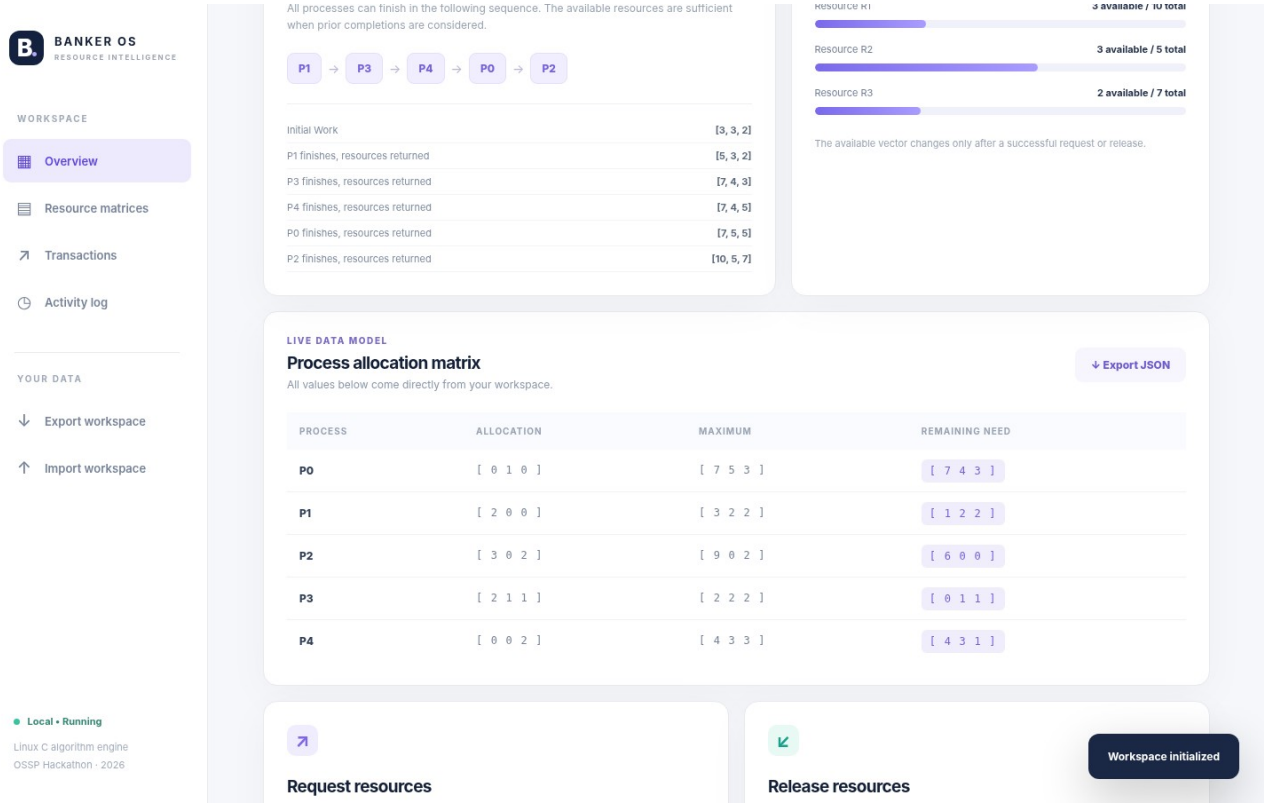
A process may release only resources it currently holds. Upon a valid release, Available increases and Allocation decreases. Need is recalculated from Maximum - Allocation. The total resource capacity remains unchanged.

7 Interactive Application Screens

First, the user enters all resource values into blank input fields. The user, not the program, chooses the process count, resource count, matrix entries and the workspace name.



The dashboard then presents real-time safety status, a safe sequence when available, resource balances, computed Need, request/release controls and an operation timeline. The screenshot below is a documented representative test dataset, not the application's default.


BANKER OS
RESOURCE INTELLIGENCE
WORKSPACE
Overview
Resource matrices
Transactions
Activity log
YOUR DATA
Export workspace
Import workspace
Local • Running
Linux C algorithm engine
OSSP Hackathon · 2026
All processes can finish in the following sequence. The available resources are sufficient when prior completions are considered.
P1 → P3 → P4 → P0 → P2
Initial Work [3, 3, 2]
P1 finishes, resources returned [5, 3, 2]
P3 finishes, resources returned [7, 4, 3]
P4 finishes, resources returned [7, 4, 5]
P0 finishes, resources returned [7, 5, 5]
P2 finishes, resources returned [10, 5, 7]
Resource R2 3 available / 5 total
Resource R3 2 available / 7 total
The available vector changes only after a successful request or release.
LIVE DATA MODEL
Process allocation matrix
All values below come directly from your workspace.
↓ Export JSON
PROCESS ALLOCATION MAXIMUM REMAINING NEED
P0 [0 1 0] [7 5 3] [7 4 3]
P1 [2 0 0] [3 2 2] [1 2 2]
P2 [3 0 2] [9 0 2] [6 0 0]
P3 [2 1 1] [2 2 2] [0 1 1]
P4 [0 0 2] [4 3 3] [4 3 1]
Request resources
Release resources
Workspace initialized

8 Testing and Validation

A controlled example with five processes and three resource types is used solely for tests and faculty demonstration. It is not loaded automatically by the browser application.

Process	Allocation	Maximum	Computed Need
P0	0 1 0	7 5 3	7 4 3
P1	2 0 0	3 2 2	1 2 2
P2	3 0 2	9 0 2	6 0 0
P3	2 1 1	2 2 2	0 1 1
P4	0 0 2	4 3 3	4 3 1

Initial Available = [3, 3, 2]. Computed safe sequence: P1 -> P3 -> P4 -> P0 -> P2.

Test input	Expected	Observed result
P1 requests [1,0,2]	Approved, state safe	GRANTED, Available [2,3,0]
P4 requests [3,3,0]	Rejected, unsafe	DENIED, rollback preserves [3,3,2]
P2 requests [4,0,0]	Denied, exceeds Available	DENIED
P1 requests [2,0,0]	Denied, exceeds Need	DENIED
P1 releases [2,0,0]	Returned to system	Available becomes [5,3,2]
Max less than Allocation	Dataset rejected	Descriptive validation error
Zero-resource edge case	Safe when Need = 0	SAFE
Multiple seeded resource sets	Invariant checks	50 randomized states checked

Automated validation: 21 unit tests passed. Additionally, direct local HTTP API checks confirmed workspace initialization, safe grant, unsafe denial with rollback, and release. A separate browser visual rendering test checked the dashboard layout and basic interface behavior. Network browser testing against localhost was blocked in the execution environment; end-to-end functionality should also be verified on the demonstration laptop.

9 Complexity and Constraints

The straightforward safety algorithm may require $O(n^2m)$ time for n processes and m resource types and uses O(nm) auxiliary storage for captured safety steps and matrices. Resource-request validation is O(m) before the safety check. Supported maximum sizes are 20 processes and 10 resource types; input values are bounded to protect the demonstration program from invalid or excessive allocations.

10 Deployment and Reproduction

```
make
```

```
make web
```

Open http://localhost:8765 on the same computer. For terminal mode use ./banker. For automated validation use make test. The application is local-only and needs GCC, make and Python 3 standard library modules; no pip install or internet connection is needed.

11 Limitations and Future Scope

- The model is a teaching simulator and does not administer real Linux kernel resources.
- The C safety checker finds one safe sequence; it does not enumerate all possible sequences.
- The dashboard is intended for one local workstation and is not a multi-user service.
- Future improvements may include step-by-step animated execution, CSV exports, process life-cycle events, and a richer desktop installer.

12 Team Participation Plan

Member	Presentation / implementation explanation
Nithish S	Problem statement, Banker core principles, interactive allocation demo
Sai Sathwik	Matrix inputs, safe-sequence steps, test scenarios and results
Ch. Vivek	Architecture, local deployment, GitHub walkthrough, documentation

The table is a suggested presentation split, not a claim that a particular person wrote a specific file. All members should understand the core algorithm and be able to explain the result.

13 Conclusion

Banker OS demonstrates how a resource manager can avoid unsafe allocations using Banker's Algorithm. User-entered matrices, clear grant/deny explanations, transactional rollback and repeatable tests make the system suitable for a live educational demonstration. The project links an understandable interface to a compiled C algorithm implementation while keeping the distinction between a simulation and the real operating system explicit.

References

1. Assigned OSSP Hackathon Problem Statements, 13 October 2026, Problem 13.
2. KLH OSSP Lab In-Sem and Hackathon Circular, 5 October 2026.
3. Operating System Concepts - standard deadlock-avoidance and Banker's Algorithm material.
4. Linux C standard library and Python standard-library documentation for implementation details.